MVC is a well-known presentation design pattern in web development, with the purpose of enforcing a separation of concerns—specifically, the concerns of the application model and its presentation.

The *V* in MVC is the view, or the page. The *M* is the application model, which is a nebulous term that represents everything in the application that isn't view or controller. The model includes the data access code, the business or domain objects—the *things* your application is all about (books, authors, and customers, in Manning's case)—and the programming logic that is designed to manage them (i.e., the business logic). The application model is then subject to further separation according to other good software design practices, but that's not the business of MVC, which is purely a presentation design pattern. The main reason for enforcing a separation between the UI and the rest of the model is improving maintenance and testability. It is very difficult to test application logic if it is mixed in with HTML.

The controller part of MVC is the primary means of separation between the model and the view. Its role is to take requests and then use the information in the request to perform commands on the model. It will then take the result of that processing and pass it to the view for display.

Controllers can be implemented in different ways. You can create something like a front controller to process requests for the whole application or a subset of the application, or you can use the page controller pattern to process the requests for individual pages. The original ASP.NET MVC framework implementation leverages the front controller approach, in which a single controller is responsible for coordinating the processing for multiple endpoints related to a feature or business area within the application—`AuthorController`, `BookController`, and so on. Razor Pages